SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
Kuniamuthur, Coimbatore. Tamil Nadu 641008
(An Autonomous Institution Affiliated to Anna University, Chennai)

**Course : Principles Of Compiler Design**

**Module I : Lexical Analysis And Syntax Analysis**

**Topic : Finite Automata**

# SESSION OUTCOMES

**At the end of the session, the learner would be able to,**

- **Represent Finite automata using different notations.**
- **Design DFA, NFA to recognise strings in a language.**

# FINITE AUTOMATA IN COMPILER DESIGN

- Regular expressions = specification of languages
- Finite automata = implementation of lexical analyzer

# FINITE AUTOMATA - Definition

- A finite automaton consists of
  - An input alphabet $\Sigma$
  - A set of states $S$
  - A start state $n$
  - A set of accepting states $F \subseteq S$
  - A set of transitions $\text{state} \xrightarrow{\text{input}} \text{state}$

- Transition

$$s_1 \xrightarrow{a} s_2$$

- Is read

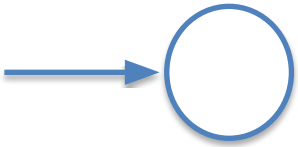
In state $s_1$ on input $a$ go to state $s_2$

- If end of input and in accepting state => accept

- Otherwise => reject

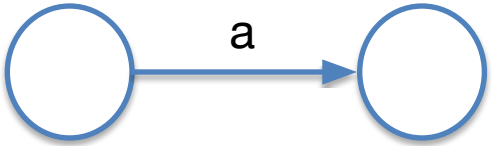
# Transition Diagram- Components

- The start state



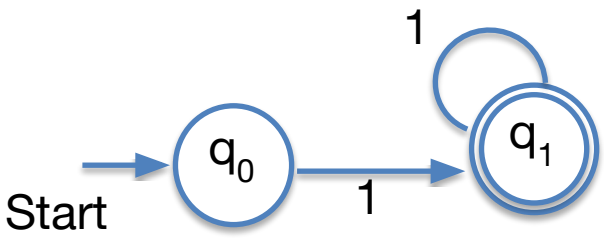
- An accepting state.

- A transition



# Deterministic Finite Automata- Examples

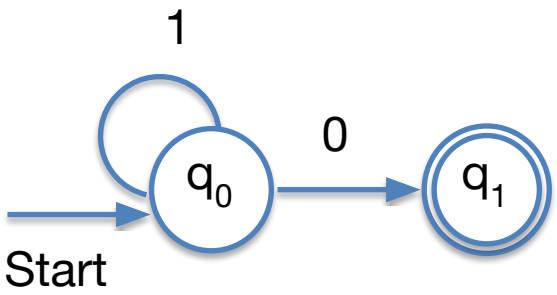
- A finite automaton that accepts only “1”




**<u>Regular Expression:</u>** $1^+$ (one or more occurrence of 1)

# Deterministic Finite Automata- Examples

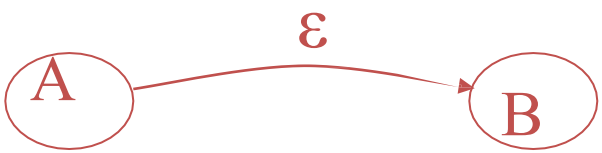
- A finite automaton accepting any number of 1's followed by a single 0
- Alphabet: {0,1}




**Regular Expression:** $1^* \cdot 0$ (zero or more occurrences of 1, followed by a zero)

# ε - Transitions

- Another kind of transition: ε-moves




- ε- Indicates the empty string
- The Finite automate indicates, transition from A to B is possible over ε

# DFA vs NFA

- Deterministic Finite Automata (DFA)
  - One transition per input per state
  - No $\varepsilon$-moves

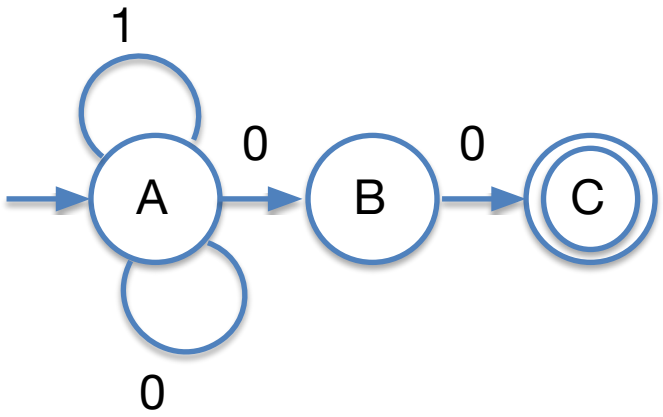
- Nondeterministic Finite Automata (NFA)
  - Can have multiple transitions for one input in a given state
  - Can have $\varepsilon$-moves

# DFA vs NFA

| DFA | NFA |
|---|---|
| One transition per input per state | Multiple transitions for one input in a given state |
| No ϵ-moves | Can have ϵ-moves |
| One path through the state graph | Multiple paths through the state graph |
| Each transition results in one state | A transition can result in a set of states (if any is final at the end of the input, accept. O.W, reject) |
| Faster to execute | Slower (exponentially) to execute |
| Exponetially large | Exponentially smaller |

# DFA vs NFA

- An NFA can get into multiple states.




| States | 0 | 1 |
|---|---|---|
| A | A, B | A |
| B | C | - |
| C | - | - |

# DFA vs NFA

- NFAs and DFAs recognize the same set of languages
  - regular languages
- DFAs are faster to execute.
  - There are no choices to consider
- NFAs are, in general, smaller.

# In Conclusion:

Finite Automata(FA) - Formalization of transition diagrams, that include,

- A state designated as start state
- One or more states, designated as accepting states.
- Input Characters.
- Transition among states.

Accepting state of a FA indicates, the lexeme for some token has been found.

FA can make transitions on empty input as well as on input characters.

# In Conclusion-cont.

Deterministic Finite Automata (DFA)

- Has exactly one transition out of each state for each input symbol.
- Transitions on empty input is not allowed.

FA that is not a DFA is a Non-deterministic Finite Automata(NFA). NFAs are easier to design than DFA.

# FOR YOU, TO THINK & SOLVE!!

DESIGN DFA & NFA.

Make use of, https://automatonsimulator.com

- All strings of digits with no repeated digits. Hint: Try this problem first with a few digits, such as {0, 1, 2}
- All strings of a's and b's with an even number of a's and an odd number of b's.
- Comments consisting of a string starting with /* and ending with */ without any */ in between unless it occurs inside balancing quotes " and ".

**Coming Up:**

Regular Expression to NFA.

NFA to DFA Conversion

Thank You!!